
Instagram Media Manager

Technical Guide — Architecture, Design, Pipelines & Debugging

Project root: `C:\laragon\www\insta_manage`

Audience: the developer maintaining and operating this application

Scope: local, single-user, self-hosted tool

Python 3.10

FastAPI

SQLAlchemy 2

MySQL 8

Celery

Redis

Instaloader

Playwright

React 18

Vite 4

Bootstrap 5

Contents

1. Architecture

1. Runtime topology — the five processes
2. Backend layering (and how it maps to MVC)
3. Request lifecycle
4. Database schema
5. The two independent auth systems

2. Design System

1. Where it came from
2. Token architecture & CSS layer order
3. Colour palette
4. Component inventory
5. The DataTable component

3. Pipelines

1. Instagram login pipeline
2. Download job pipeline
3. Media-type resolution
4. Storage layout & metadata capture

4. Debugging Guide

1. Service health checklist
2. Where the logs are
3. Failure catalogue — symptom → cause → fix
4. Inspecting state with SQL
5. Testing the API directly
6. Frontend debugging (incl. headless screenshots)
7. Case studies: three real bugs
8. Reset procedures

5. Appendix — Command Reference

1. Architecture

1.1 Runtime topology — the five processes

The application is not a single program. Five processes must be running; four of them are long-lived services and one is the dev server for the UI. Most "nothing happens when I click download" reports trace back to one of these being down.

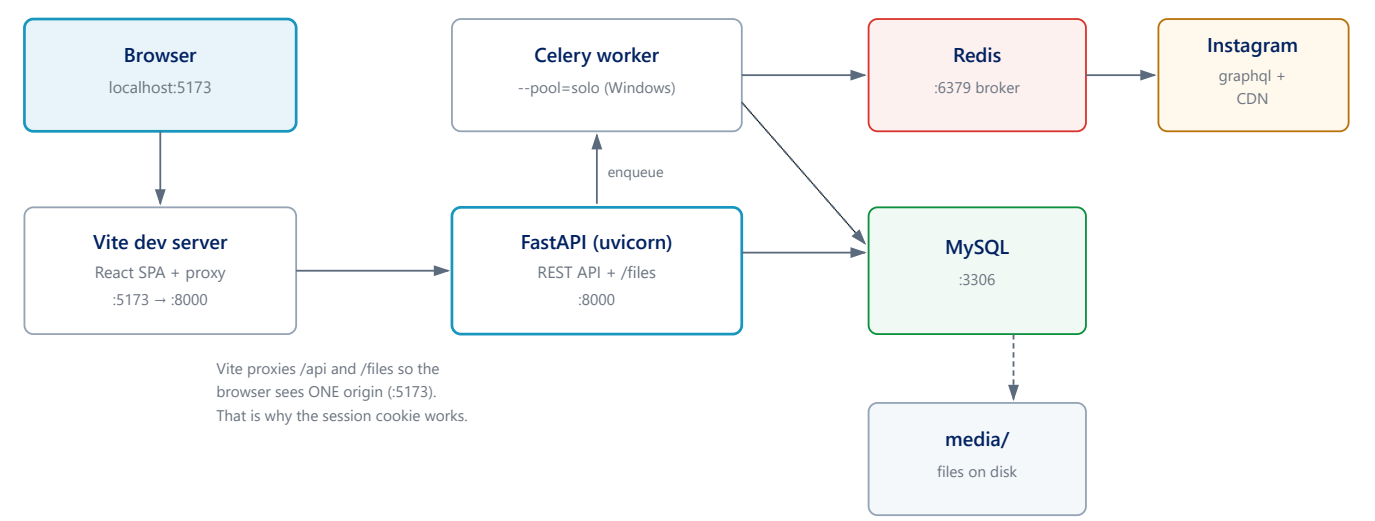


Figure 1 — Runtime topology. Dashed lines are writes performed by the Celery worker.

Process	Start command	If it is down
MySQL	Laragon, or <code>mysqld.exe</code>	Backend fails on startup / every request 500s
Redis	<code>redis-server.exe --port 6379</code>	API still works; jobs never start (stay <code>pending</code>)
Celery worker	<code>celery -A app.tasks.celery_app worker --pool=solo</code>	Jobs queue up but never execute
FastAPI	<code>uvicorn app.main:app -- reload</code>	UI loads but every call fails
Vite	<code>npm run dev</code>	No UI at all

Windows-specific
Celery's default `prefork` pool does not work on Windows. `--pool=solo` is mandatory — without it the worker starts, connects, prints "ready", and then silently never executes a task.

1.2 Backend layering (and how it maps to MVC)

The backend is a layered architecture. It borrows from MVC but is not textbook MVC, because the server renders no HTML — the "view" is a separate React app consuming JSON.

Layer	Directory	Responsibility
Model	<code>app/models/</code>	SQLAlchemy ORM classes — the actual tables. <i>Same role as MVC's Model.</i>
Controller	<code>app/routers/</code>	HTTP endpoints. Parse/validate, call a service, shape the response. Deliberately thin.
View	<i>(none server-side)</i>	Replaced by the React SPA. This is the main divergence from classic MVC.
Service	<code>app/services/</code>	Business logic: Instagram access, Playwright login, crypto, password hashing. Not an MVC concept — added so controllers stay thin and logic is reusable from Celery tasks.
Schema (DTO)	<code>app/schemas/</code>	Pydantic request/response contracts. Gives automatic validation and the OpenAPI docs.
Task	<code>app/tasks/</code>	Celery entry points. Own transaction + job-status bookkeeping.

Why services exist as their own layer

`download_posts()` is called from a Celery task, never from a router. If that logic lived in the router it would be unreachable from the worker. Keeping it in `services/` means the same function serves both entry points.

1.3 Request lifecycle

```
# Example: POST /api/downloads/jobs
Browser → Vite proxy → FastAPI
|
|─ CORS middleware
|─ protect_media_files middleware # only /files/*
|─ Depends(get_current_admin)    # reads httpOnly cookie
|─ Pydantic validates DownloadJobCreate
|─ router: downloads.create_job()
|   |─ INSERT download_jobs (status=pending)
|   |   └─ task.delay(job.id) → Redis
|   └─ 201 + DownloadJobRead
└─ Celery worker ← Redis
    |─ _run_job(): status=running, started_at=now
    |─ get_authenticated_loader(db) # decrypt IG cookies
    |─ instaloader_service.download_*()
    └─ status=success|failed, finished_at=now
```

1.4 Database schema

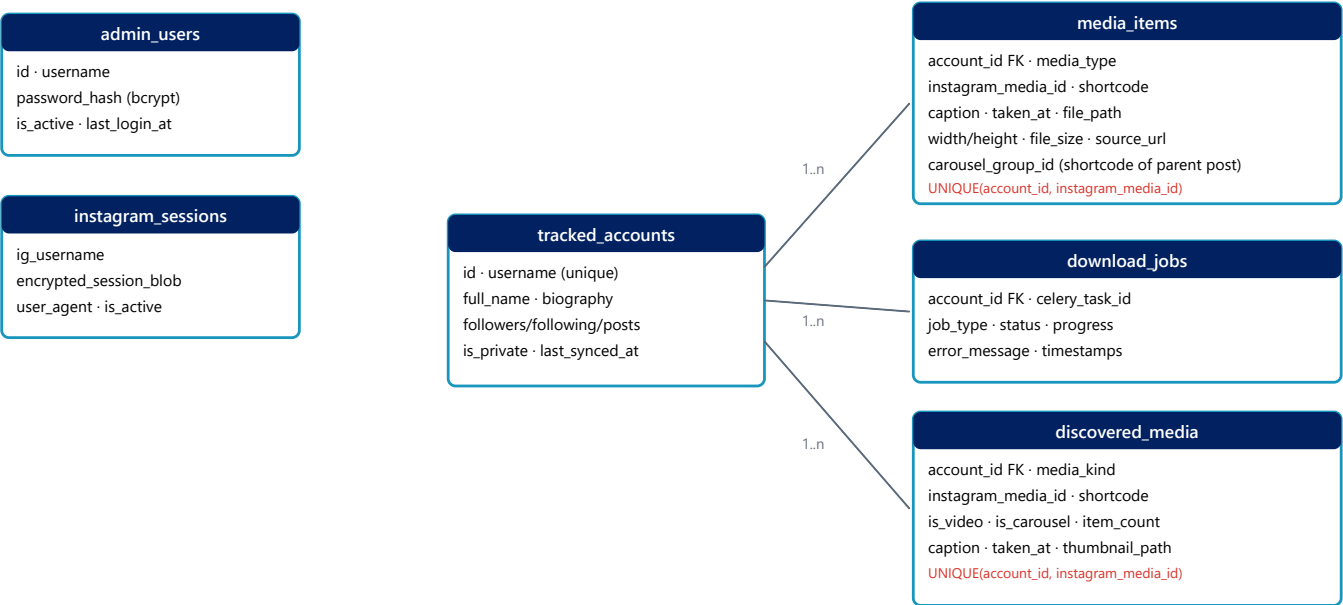


Figure 2 — Schema. All three child tables cascade-delete with their account. discovered_media holds listings only (no media bytes) — see §3.4.

The idempotency key
UNIQUE(account_id, instagram_media_id) is what makes re-running a download safe — on both media_items and discovered_media. Carousel children get a synthetic id of {mediaid}_{index} so each slide is tracked separately, and share the parent post's shortcode as carousel_group_id so the UI can group them back together.

1.5 The two independent auth systems

These are frequently confused. They are unrelated and fail differently.

	Back-office session	Instagram session
Protects	This application	Nothing — it grants access to Instagram
Stored in	admin_users (bcrypt hash)	instagram_sessions (Fernet-encrypted cookies)
Transport	httpOnly signed cookie imm_session	Server-side only; never sent to the browser
Failure looks like	Login screen appears; 401 on every route	"No active Instagram session" in a job's error column
Fix	Sign in, or re-run scripts.create_admin	Re-run the browser login flow

2. Design System

2.1 Where it came from

The visual language was ported from the `desmondypap` admin theme (`C:\laragon\www\desmondypap\adm`) — specifically `dashboard/inc/color-dark-theme.php` and `inc/header-cons.php` . The source is PHP + jQuery, so files could not be copied verbatim; the *design decisions* were re-expressed as CSS custom properties and React components.

Source (desmondypap)	Destination (this app)
Hard-coded hex values across PHP includes	<code>styles/tokens.css</code> — CSS variables
<code>.sidebar-d</code> , <code>#sb-active</code> , <code>#bc-div</code>	<code>styles/layout.css</code> + <code>components/layout/</code>
<code>.btn1btn6</code> , <code>#general-btn</code>	<code>.btn-gen</code> + <code>--primary/--success/--danger/--navy/--info</code>
<code>.tb-ac-btn</code> , <code>.tb-status</code> , <code>.general-box</code>	<code>.ac-btn</code> , <code>.pill</code> , <code>.panel</code>
jQuery DataTables + 10 JS files	<code>components/datatable/DataTable.jsx</code>
<code>icons.css</code> (678 KB, 5 icon sets)	npm <code>remixicon</code> (1 set, versioned)

Why DataTables was rewritten rather than imported

jQuery DataTables takes ownership of the DOM it renders. React also owns that DOM and re-renders rows whenever state changes. The two fight, and rows detach or duplicate. The React port keeps the identical look and features (search, page-length, sorting, info line, pagination, CSV/print) without the conflict.

2.2 Token architecture & CSS layer order

Everything visual resolves to `styles/tokens.css` . No component file contains a raw hex colour. Import order matters and is fixed in `styles/index.css` :

```
tokens.css    ← variables – must be first
base.css      ← resets, typography, scrollbars
layout.css    ← sidebar / topbar / breadcrumb shell
components.css ← buttons, panels, pills, forms
datatable.css ← table, toolbar, pagination
media.css     ← gallery grid, thumbnails, modal
login.css     ← the sign-in gate
```

Consequence: re-theming the entire application — back office and any future front-facing page — is a single-file edit.

2.3 Colour palette

Token	Value	Applied to
--c-primary	 #1896bd	Buttons, links, panel titles, focus rings
--c-mint	 #24ecc5	Gradient terminus, active sort arrow
--grad-primary	58° teal→mint	Active sidebar item, avatars, login header
--c-navy-deep	 #000f2d	Sidebar background
--c-navy	 #012161	Table headers, active pagination
--c-body-bg	 #eaedf7	Page background
--c-success	 #06d736	Public badge, success pill
--c-danger	 #ff4d4d	Delete buttons, failed jobs, errors
--c-info	 #00b7ff	Export buttons, edit action, running jobs
--c-warning	 #e18d00	Private badge
--shadow-card	0 10px 30px /5%	Every raised surface
--lift	translateY(-5px)	Signature hover on buttons & cards

2.4 Component inventory

```
components/
├─ layout/    AppLayout  shell + mobile drawer state
│             Sidebar    nav defined as DATA, not JSX
│             Topbar     IG status + admin identity/logout
│             Breadcrumb trail=[{label,to}]
├─ ui/        Panel · StatusPill · EmptyState · ConfirmModal
├─ datatable/ DataTable
└─ media/     MediaCard · MediaModal
```

Adding a nav item is one line in `NAV_SECTIONS` inside `Sidebar.jsx`; no JSX edit required.

2.5 The DataTable component

```
<DataTable
  columns=[
    { key: 'username', header: 'Account', sortable: true,
      value: (r) => `${r.username} ${r.full_name} | |`, // search/sort/export text
      render: (r) => <AccountCell row={r} /> }, // what is displayed
    { key: 'action', header: 'Action', searchable: false,
      render: (r) => <ActionButtons row={r} /> },
  ]
  rows={accounts}
  exportable
  exportFilename="tracked-accounts"
  defaultPageSize={10}
/>
```

Column key	Meaning
<code>render(row,i)</code>	JSX shown in the cell. <code>i</code> is the absolute index in the sorted set, so row numbers stay correct across pages.
<code>value(row)</code>	Plain text used for sorting, searching and export. Required whenever <code>render</code> shows more than one field , otherwise search silently ignores the extra text.
<code>sortable</code>	Enables click-to-sort. Numeric values compare numerically; strings use <code>localeCompare</code> with <code>numeric:true</code> .
<code>searchable:false</code>	Excludes the column from the search filter (row numbers, action buttons).

The most likely DataTable mistake

Omitting `value` on a composite cell. The column renders fine, but searching for text that is visibly on screen returns nothing — because the filter only saw `row[key]`. This exact bug shipped and was caught in testing (see §4.7).

3. Pipelines

3.1 Instagram login pipeline

Design constraint: **the application must never receive the Instagram password.** This is achieved by not building a password field at all — a real browser is opened on Instagram's own login page and only the resulting cookies are harvested.

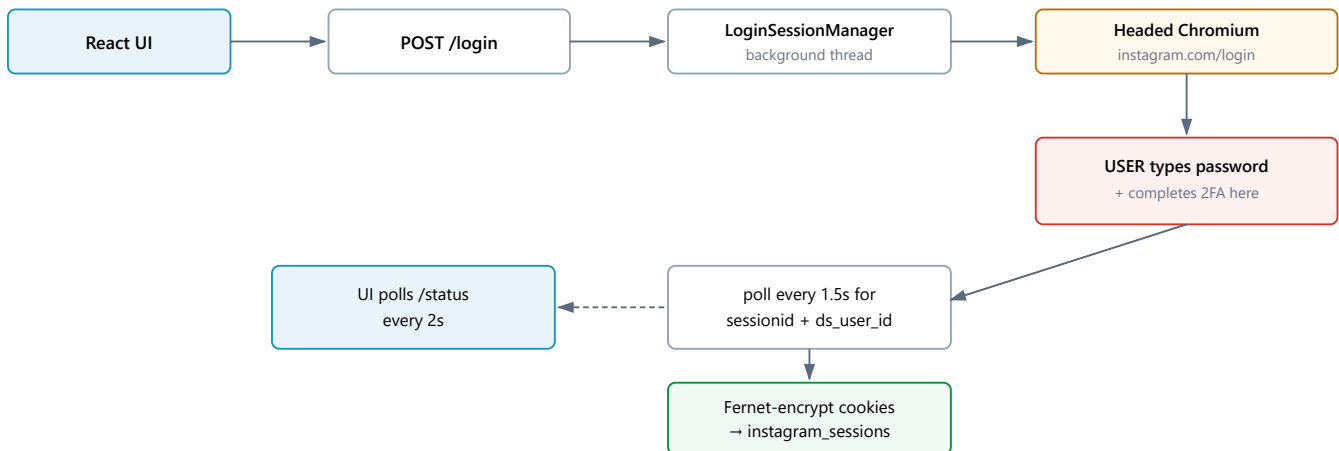


Figure 3 — Login pipeline. The password exists only inside the Chromium window, on Instagram's own page.

Implementation notes worth knowing when debugging:

- It runs in a **plain Python thread**, not Celery — it needs a live desktop window and 1.5-second polling, which a distributed worker cannot provide.
- Success is detected by the presence of both `sessionid` and `ds_user_id` cookies — not by URL matching, which is brittle across Instagram's redirect variants.
- The captured User-Agent is stored *alongside* the cookies and replayed on every later request. Mismatching the UA is a common cause of instant session invalidation.
- Timeout is `LOGIN_TIMEOUT_SECONDS` (default 300). Closing the window early yields status `closed`, not `failed`.
- Terminal states: `connected` · `failed` · `timed_out` · `closed`. The manager's state lives in memory, so a backend restart mid-login abandons the attempt.

3.2 Download job pipeline

```
POST /api/downloads/jobs
├─ INSERT download_jobs (status=pending, progress 0/0)
├─ task.delay(job_id) ─► Redis ─► Celery worker
│
│   ┌─ _run_job(job_id, action) ───────────┐
│   │   ┌─ status = running, started_at = now()
│   │   │   ┌─ get_authenticated_loader(db)
│   │   │   │   ┌─ SELECT active instagram_sessions
│   │   │   │   │   ┌─ Fernet-decrypt cookie blob
│   │   │   │   │   │   ┌─ rebuild requests.Session (cookies + stored UA)
│   │   │   │   │   │   │   ┌─ loader.test_login() # raises if session died
│   │   │   │   │   │   │   │   ┌─ action(db, loader, account, progress_cb)
│   │   │   │   │   │   │   │   │   ┌─ per item: resolve URL → context.get_raw() → write bytes
│   │   │   │   │   │   │   │   │   │   ┌─ → probe dimensions → UPSERT media_items
│   │   │   │   │   │   │   │   │   │   │   ┌─ → progress_cb(i, total) → COMMIT
│   │   │   │   │   │   │   │   │   │   │   ┌─ → sleep(DOWNLOAD_REQUEST_DELAY_SECONDS)
│   │   │   │   │   │   │   │   │   │   │   └─
│   │   │   │   │   │   │   │   └─ status = success | failed(+error_message), finished_at = now()
│   │   │   │   │   │   │   └─
│   │   │   │   │   │   └─
│   │   │   │   │   └─
│   │   │   │   └─
│   │   │   └─
│   │   └─
│   └─
```

Job type	What it iterates
<code>sync_profile</code>	Profile metadata + avatar only. Cheapest; use it to test connectivity.
<code>download_posts</code>	<code>profile.get_posts()</code> — every post, expanding carousels
<code>download_reels</code>	Same, filtered by <code>_is_reel()</code>
<code>download_stories</code>	<code>get_stories()</code> → <code>story.get_items()</code> (24h window only)
<code>download_single_post</code>	One shortcode. Best first end-to-end test.

Why `context.get_raw()` rather than plain `requests`

It reuses Instaloader's authenticated session *and* its retry/back-off handling, so the download inherits polite behaviour toward Instagram's rate limits instead of reimplementing it badly.

3.3 Media-type resolution

A single Instagram post can produce one row or twelve. The branching in `_post_to_pending()` :

```
if post.type == 'GraphSidecar':      # carousel
    for i, node in enumerate(post.get_sidecar_nodes()):
        type = CAROUSEL_VIDEO if node.is_video else CAROUSEL_IMAGE
        id = f'{post.mediaid}_{i}'    # synthetic per-slide id
        url = node.video_url if node.is_video else node.display_url
elif post.is_video:
    type = REEL if _is_reel(post) else POST_VIDEO
    url = post.video_url
else:
    type = POST_IMAGE
    url = post.url
```

Known limitation — reel detection

Instaloader's `Post` class exposes no `product_type` property, and Instagram's timeline GraphQL response does not reliably include one. `_is_reel()` reads `post._node.get('product_type') == 'clips'` when present and otherwise classifies the item as `post_video`.

"Download Reels" may therefore under-count.

If reels appear missing, this is the first place to look — not the network layer.

3.4 Storage layout & metadata capture

```
media/  
└─ {username}/  
   └─ profile_pic.jpg  
   └─ post_image/ 20240115_142233_CxYz123AbCd.jpg  
   └─ carousel_image/ 20240115_142233_CxYz123AbCd.jpg  
   └─ reel/  
   └─ story_video/
```

Filename pattern: `{taken_at:%Y%m%d_%H%M%S}_{shortcode|media_id}.{ext}`. The extension is derived from the HTTP `Content-Type`, falling back to the URL suffix.

Files are written byte-for-byte as served. Pillow is used only to *read* width/height from the in-memory buffer — the image is never re-encoded or resized. Captured per item: media type, Instagram media id, shortcode, caption, `taken_at`, download timestamp, path, extension, resolution, byte size, and source URL.

4. Debugging Guide

4.1 Service health checklist

Run this first, every time. It isolates the failing layer in about ten seconds.

```
# 1. MySQL
C:\laragon\bin\mysql\mysql-8.0.30-winx64\bin\mysqladmin.exe -u root ping
#   → "mysqld is alive"

# 2. Redis
C:\laragon\bin\redis\redis-x64-5.0.14.1\redis-cli.exe -p 6379 ping
#   → "PONG"

# 3. FastAPI
curl http://127.0.0.1:8000/api/health
#   → {"status":"ok"}

# 4. Vite
curl -o NUL -w "%{http_code}" http://127.0.0.1:5173/
#   → 200

# 5. Celery — must print "celery@HOST ready." and list 5 tasks
```

4.2 Where the logs are

Source	Where to look
FastAPI	The uvicorn terminal. Every request logs method, path, status.
Celery task failures	The worker terminal and <code>download_jobs.error_message</code> (also shown in the Jobs page).
Playwright login	Uvicorn terminal (runs in-process). Failures surface via <code>GET /login/{id}/status</code> .
Frontend	Browser DevTools → Console and Network.
MySQL	<code>C:\laragon\data\mysql-8*.err</code>

Persist a log to file

```
uvicorn app.main:app > uvicorn.log 2>&1 — otherwise output is lost when the terminal closes.
```

4.3 Failure catalogue — symptom → cause → fix

Symptom	Cause	Fix
Login screen keeps returning; <code>401</code> on every <code>/api/*</code>	No/expired <code>imm_session</code> cookie, or <code>SECRET_KEY</code> changed (which invalidates all sessions)	Sign in again. If credentials are lost, re-run <code>python -m scripts.create_admin</code>
Job stuck at <code>pending</code> forever	Celery worker or Redis is not running	Start both. Confirm the worker prints <code>ready</code> .
Worker says <code>ready</code> but tasks never execute	Prefork pool on Windows	Restart with <code>--pool=solo</code>
"No active Instagram session" in a job error	Never connected, session expired, or it was deactivated	Instagram Login page → Connect
"Instagram session is no longer valid"	<code>test_login()</code> failed — cookies revoked by Instagram	Re-run the browser login
<code>401 ... "Please wait a few minutes"</code>	Instagram rate-limited the account/IP	Wait. Raise <code>DOWNLOAD_REQUEST_DELAY_SECONDS</code> . Do not retry in a loop.
Images broken in the library; <code>/files/*</code> returns 401	Not signed in — the media mount is auth-gated by middleware	Sign in. If it persists, confirm the cookie is being sent.
Media library grid empty but files exist on disk	Rows missing from <code>media_items</code> (job failed mid-way)	Check the job's error; re-run with force re-download
"Download Reels" finds far fewer than expected	<code>product_type</code> absent from the API response (§3.3)	Use "Download All Posts" — reels are included as <code>post_video</code>
Private account returns nothing	The authenticated account does not follow it	Expected. Access is limited to what the session may legitimately see.
Vite: <code>crypto\$2.getRandomValues</code> is not a function	Vite 5 requires a newer Node than Laragon's 18.8	Stay on Vite 4 (already pinned), or upgrade Node
Backend won't start: <code>Can't connect to MySQL</code>	MySQL not running	Start it from Laragon
Alembic: Target database is not up to date	Migrations not applied	<code>python -m alembic upgrade head</code>

4.4 Inspecting state with SQL

```
-- Is a session connected, and to whom?
SELECT ig_username, is_active, updated_at FROM instagram_sessions ORDER BY id DESC;

-- Recent jobs with failures
SELECT id, job_type, status, progress_current, progress_total,
       LEFT(error_message,90) AS err, created_at
FROM download_jobs ORDER BY id DESC LIMIT 20;

-- What actually got downloaded, by type
SELECT a.username, m.media_type, COUNT(*) AS n, SUM(m.file_size_bytes)/1048576 AS mb
FROM media_items m JOIN tracked_accounts a ON a.id = m.account_id
GROUP BY a.username, m.media_type;

-- Rows whose file is referenced but may be missing on disk
SELECT id, file_path, file_size_bytes FROM media_items WHERE file_size_bytes IS NULL OR file_size_bytes = 0;

-- Confirm no plaintext password is stored
SELECT username, LEFT(password_hash,7) AS algo, LENGTH(password_hash) AS len FROM admin_users;
-- expect: $2b$12$ / 60
```

4.5 Testing the API directly

Bypassing the UI isolates backend from frontend. FastAPI serves interactive docs at

http://127.0.0.1:8000/docs; log in through the UI first so the browser holds the cookie, then call endpoints from that page.

```
# From the shell — keep a cookie jar
curl -c c.txt -X POST http://127.0.0.1:8000/api/admin/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"..."}'

curl -b c.txt http://127.0.0.1:8000/api/accounts
curl -b c.txt http://127.0.0.1:8000/api/downloads/jobs

# Queue a job without the UI
curl -b c.txt -X POST http://127.0.0.1:8000/api/downloads/jobs \
  -H "Content-Type: application/json" \
  -d '{"account_id":1,"job_type":"sync_profile"}'
```

Delete the cookie jar afterwards

`c.txt` holds a valid 7-day session token. Remove it when finished.

4.6 Frontend debugging

- **Network tab** — confirm the request went to `/api/...` and that `imm_session` is attached. A `401` here means auth, not application logic.
- **Console** — React surfaces key warnings and prop errors here.
- **React StrictMode double-invokes effects in development.** Seeing two identical requests on mount is expected and does not occur in a production build.
- **The cookie is invisible to JavaScript by design.** `document.cookie` returning `''` is correct — verify the cookie in DevTools → Application → Cookies instead.

Headless screenshot debugging

Playwright is already installed for the login flow and can also drive the UI. This catches rendering and interaction faults that reading code does not:

```
# backend\venv\Scripts\python screenshot.py
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    b = p.chromium.launch()
    pg = b.new_page(viewport={"width":1400,"height":950})
    pg.on("pageerror", lambda e: print("JS ERROR:", e))
    pg.on("console", lambda m: m.type=="error" and print("CONSOLE:", m.text))
    pg.goto("http://127.0.0.1:5173/accounts", wait_until="networkidle")
    pg.fill("#username","admin"); pg.fill("#password","...")
    pg.click("button[type=submit]"); pg.wait_for_timeout(1500)
    pg.screenshot(path="out.png", full_page=True)
    b.close()
```

4.7 Case studies: three real bugs

All three passed code review and were only exposed by exercising the running application.

1 — Route collision made the Media Library unreachable

The backend mounted static files at `/media`; the SPA also routed `/media` to the Media Library page. Vite proxied the path to the backend, which returned `{"detail":"Not Found"}` instead of rendering the page.

Detection:

a screenshot showing raw JSON where the page should be.

Fix:

static mount moved to `/files`.

Lesson:

API paths and client-side routes share one namespace once a proxy is involved.

2 — Pagination silently reset to page 1

`updateParam()` set the requested key, then unconditionally executed `next.set('page', '1')`. Because page changes flowed through the same helper, every page click was overwritten back to 1.

Detection:

asserting the info line after clicking page 2 — expected "11 to 12", got "1 to 10".

Fix:

split into `updateFilter()` (resets page) and `goToPage()` (does not).

Lesson:

one helper serving two intents will eventually serve one of them wrongly.

3 — Search ignored visible text

The Account cell rendered username *and* full name, but the column had no `value` accessor, so the filter only saw `row.username`. Searching "space" matched `spacex` but not "European Space Agency".

Detection:

comparing result count against what was visibly on screen.

Fix:

```
value: (r) => `${r.username} ${r.full_name}`.
```

Lesson:

when display and data diverge, search must follow display.

4.8 Reset procedures

```
# Forgot the back-office password
# edit ADMIN_PASSWORD in backend\.env, then:
venv\Scripts\python -m scripts.create_admin

# Force a fresh Instagram login
UPDATE instagram_sessions SET is_active = 0;

# Clear job history (safe – does not touch downloaded media)
DELETE FROM download_jobs;

# Remove one account and everything it owns (FK cascade)
DELETE FROM tracked_accounts WHERE username = 'someuser';
# files under media\someuser\ must be deleted manually

# Rebuild the schema from scratch (DESTROYS ALL DATA)
DROP DATABASE instagram_media_manager;
CREATE DATABASE instagram_media_manager CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
# then: alembic upgrade head && python -m scripts.create_admin
```

Rotating `SECRET_KEY` has two effects

It signs back-office sessions *and* encrypts the stored Instagram cookies. Changing it logs out every admin session **and**

makes the saved Instagram session undecryptable — a fresh browser login will be required.

5. Appendix — Command Reference

Start everything (four terminals)

```
-- 1 -- C:\laragon\bin\redis\redis-x64-5.0.14.1\redis-server.exe --port 6379

-- 2 -- cd backend
      venv\Scripts\python -m celery -A app.tasks.celery_app worker --pool=solo --loglevel=info

-- 3 -- cd backend
      venv\Scripts\python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

-- 4 -- cd frontend
      "C:\laragon\bin\nodejs\node-v18\npm.cmd" run dev

MySQL is started from Laragon.  UI: http://localhost:5173
```

Database

```
venv\Scripts\python -m alembic upgrade head           # apply migrations
venv\Scripts\python -m alembic revision --autogenerate -m "msg"
venv\Scripts\python -m alembic current               # show revision
venv\Scripts\python -m alembic downgrade -1          # roll back one
venv\Scripts\python -m scripts.create_admin          # seed/reset admin
```

Frontend

```
npm run dev      # dev server, HMR
npm run build    # production build – run this to catch JSX/import errors
npm run preview  # serve the built bundle
```

Key file locations

Path	Contains
backend\.env	Secrets. Gitignored. DB URL, SECRET_KEY, admin bootstrap creds
backend\app\services\instaloader_service.py	All Instagram fetching and file writing
backend\app\services\instagram_login.py	Playwright login thread
backend\app\services\security.py	bcrypt, session signing, login throttle
backend\app\tasks\download_tasks.py	Celery tasks + job status bookkeeping
frontend\src\styles\tokens.css	Every colour in the application
frontend\src\components\datatable\DataTable.jsx	The reusable table
media\	Downloaded originals. Gitignored.

Operating boundaries

This application deliberately does **not** implement CAPTCHA solving, 2FA bypass, private-account circumvention, or rate-limit evasion. Access is bounded by what the authenticated Instagram session is legitimately permitted to see. `DOWNLOAD_REQUEST_DELAY_SECONDS` exists as courtesy throttling; increasing it is the correct response to rate limiting.